VRIJE
UNIVERSITEIT
BRUSSEL

IMPLEMENTING A BENCHMARK IN BENCHKIT

Performance-Analysis-and-Evaluation

Gérard Lichtert
0557513
gerard.lichtert@vub.be

May 10, 2026

Sciences and Bio-Engineering Sciences

Contents

1 Introduction

For this assignment I chose to implement the memcached benchmark. memtier_benchmark is a high-performance traffic generation tool specifically designed to stress-test NoSQL key-value stores like Redis, Valkey, and Memcached. While many benchmarks focus on how fast a database can write to a disk, memtier focuses on high-throughput network I/O and concurrency. It simulates thousands of simultaneous clients to measure how well a database handles a massive volume of requests and at what point the response time (latency) begins to degrade. It is used to check if a new release has not decreased in performance. The source code for the benchmark and campaign can be found at *benchkit/benches/memcached/benchmark* and *benchkit/benches/memcached/campaign* respectively.

2 Parameters

The passable parameters to the benchmark are:

- server_port: server port
- protocol: type of key-value store
- authenticate: authentication values
- cluster_mode: run client in cluster mode
- requests: amount of requests or all keys
- nb_clients: Amount of clients per thread
- nb_threads: Amount of threads
- run_count: Amount of runs (unsupported for collection)
- test_time: Limit run by test_time
- pipeline: Number of concurrent pipelined requests
- ratio: Ratio of Sets:Gets
- key_pattern: Key pattern for the command
- data_size: Object size in bytes
- key_minimum: Key ID minimum value
- key_maximum: Key ID maximum value

Whilst these are not all the possible parameters that can be passed to *memtier_benchmark*, these are the most interesting. I varied my threads from 1 to 12, applied a gaussian distribution to the key_pattern and repeated the campaign 30 times (not to be confused with run_count)

3 Captured results

The benchmark not only captures the operations per second, hits per second, misses per second, but also Average latency, p50 latency, p99 latency, p99.99 latency and KB per second. It records these for the sets, gets, waits and the total.

4 Campaign

The implemented campaign measures the values stated in 3 as well as the amount of instructions, cache misses and cycles per run. I used a *PerfStatWrapper* and I chose it to try to see if there was a corrolation between cache misses and amount of threads used, as well as to see what the IPC is.

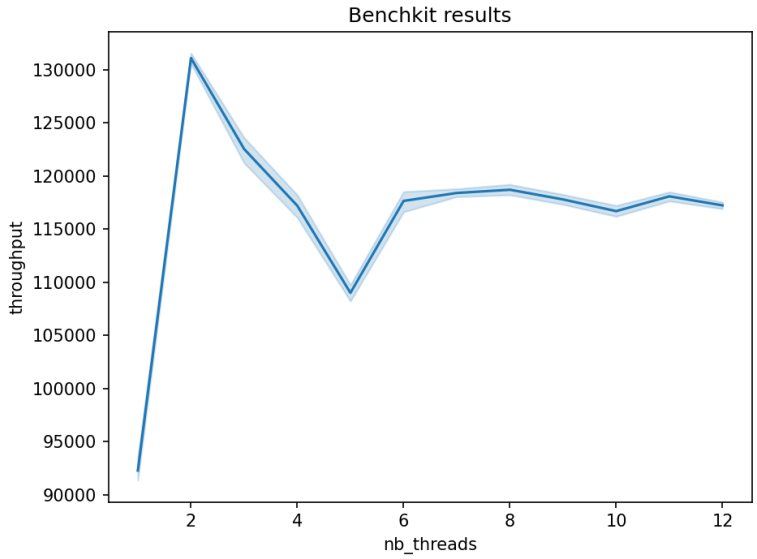

Figure 1: throughput given N threads

In 1 we can see that the throughput rises at first but then drops before stabilizing. Whilst there is an optimization to be made by using more than one thread, it seems that there is a limit in how much performance can be gained by adding threads, as per Amdahls law. Notably however, the speedup is quite poor starting from a low amount of threads, likely indicating that it's very tough to parallelize or that the majority is sequential.

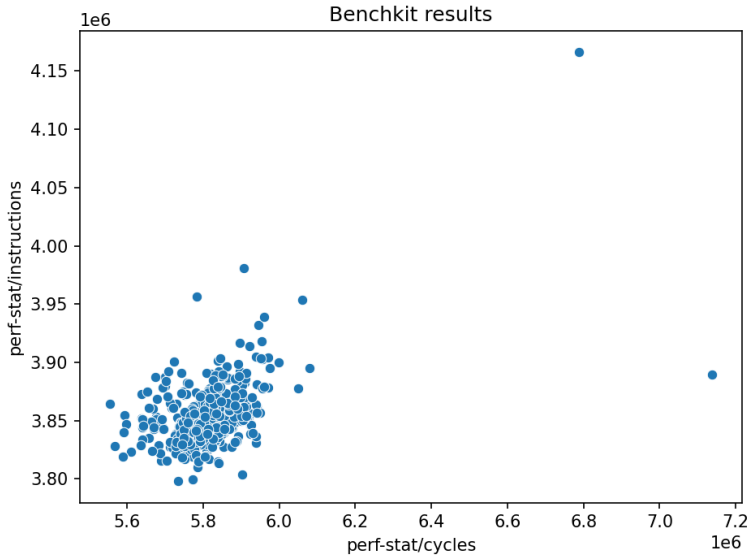

Figure 2: Instructions given n cycles

in 2 we can see that about 3.85 million instructions are executed during roughly 5.8 million cycles. This is roughly an 0.66 IPC which is relatively low. This could indicate that the CPU spends a significant amount of time doing nothing. Likely due to it having to wait for data from memory due to cache misses. This is not unusual for database related workloads (such as the key value store this benchmark operates on)

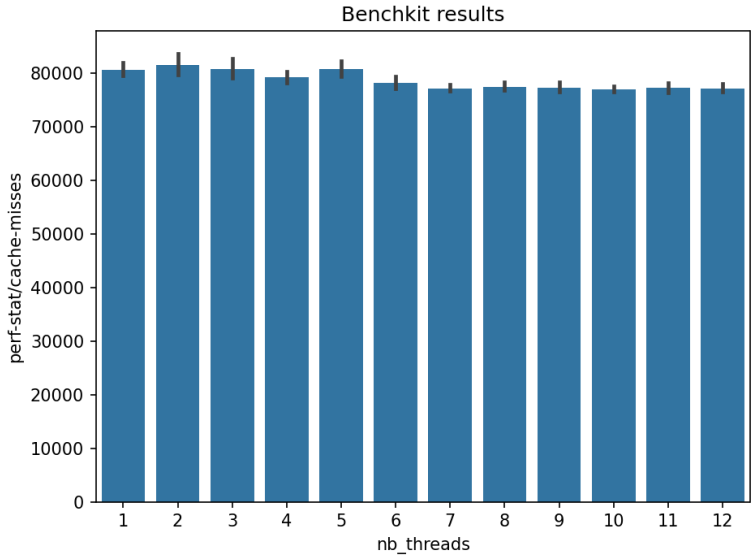

Figure 3: cache misses given n threads

using both 1 and 3 we can actually see that of the $\approx$ 115K operations per second (if we assume a time elapsed of around 10 seconds), there aren't many cache misses. However, it would perhaps be more interesting to verify which cache is being missed, as missing in a lower level cache is more expensive than in a higher one. Regardless, it's not surprising that the IPC is so low, since it is mainly working with DRAM, which is orders of magnitudes slower than memory in the caches.